

Joint Calibration and Cascade Algorithms

18.1 The problem in full

Chapters 16 and 17 built the two halves separately: strip and fit caplet vols, then use them (frozen, as an input) to match swaption vols by searching over correlation and any remaining free volatility parameters. In practice, a production calibration rarely runs these as two fully independent stages — the caplet fit and the swaption fit interact, because the same underlying instantaneous volatility parameters (a, b, c, d from Chapter 14) appear inside *both* the caplet pricing formula and the swaption approximation formula of Chapter 17. This chapter covers the two dominant ways the industry handles that interaction: cascade calibration, which imposes a specific solving order and exploits the structure of the swaption grid; and joint (simultaneous) optimization, which treats the whole problem as one nonlinear least-squares fit.

18.2 Cascade calibration

The name comes from the specific, deliberately ordered way the swaption grid is walked through, exploiting the fact that a co-terminal swaption (one whose underlying swap ends on the model's final tenor date) has a particularly simple relationship to the swaptions immediately senior to it on the grid.

Key result: the cascade idea

Process co-terminal swaptions — those all ending at the same final date T_N , with increasingly short expiries $T_{N-1}, T_{N-2}, \dots$ — in order from the *longest* expiry (shortest underlying swap) down to the *shortest* expiry (longest underlying swap). At each step, all but one of the instantaneous volatilities needed to price that swaption under Chapter 17's approximation formula have already been pinned down by previous steps; the current step's equation is therefore (approximately) a single new unknown in one new equation, solvable directly rather than through a full multi-dimensional search. Two named variants govern how the correlation and remaining volatility parameters are held or adjusted during this walk: the **triangular cascade**, which fixes correlation in advance (from historical data or Chapter 14's parametric form) and solves only for volatility at each step; and the **rectangular cascade**, which extends the same walk across every expiry/tenor combination on the grid, not just the co-terminal diagonal, re-using already-solved values as it proceeds.

ON THE DESK

Cascade calibration's genuine appeal on a desk is speed and diagnostic transparency: each step is a small, well-behaved root-find rather than a large nonlinear search, so it runs fast enough to recalibrate live, intraday, and — critically for a production system — when a fit goes wrong, the cascade's step-by-step structure makes it comparatively easy to identify *which specific instrument* the model can't reconcile with everything already

fitted, rather than receiving a single opaque “optimizer did not converge” failure from a black-box global search. This diagnostic property is, in practice, worth almost as much as the speed on a desk that has to explain a bad calibration to a risk manager within minutes of it happening.

PITFALL

Cascade calibration’s step-by-step character is also its main weakness: because each step treats previously-solved parameters as fixed and exact, small pricing or data errors introduced early in the cascade — at the longest-expiry, shortest-tenor end — propagate forward and accumulate through every subsequent step, with no mechanism for a later step to “correct” an earlier one. A cascade calibration that fits the first few swaptions on the grid beautifully and then shows steadily growing residuals as it works through the grid is very often exhibiting exactly this error accumulation, not a genuine breakdown of the model at the far end of the grid. This is the single most cited reason production systems increasingly favour joint optimization, or at minimum use a completed cascade fit purely as a fast, well-behaved *starting guess* for a subsequent joint refinement rather than as the final answer.

18.3 Joint (global) optimization

The alternative is to stop solving sequentially and instead pose the entire calibration as one nonlinear least-squares problem: find the parameter vector θ (Rebonato a, b, c, d , correlation-form parameters, any displacements) that minimises the total, appropriately weighted, squared distance between every model-implied price (or vol) and every corresponding market quote, across caps and swaptions simultaneously.

Key result: the joint calibration objective

$$\theta^* = \arg \min_{\theta} \sum_k w_k \left(\sigma_k^{\text{market}} - \sigma_k^{\text{model}}(\theta) \right)^2 + \lambda R(\theta)$$

where the sum runs over every calibration instrument k (caplets and swaptions together), w_k are weights (commonly vega-based, so that instruments with more genuine price sensitivity to volatility — and therefore more reliable implied information — are trusted more), and $R(\theta)$ is an optional **regularization** term (e.g. penalising large deviations of θ from a prior or a previous day’s calibrated values) with strength λ , added specifically to control the ill-posedness discussed below.

The standard numerical workhorse for this kind of problem is the **Levenberg–Marquardt algorithm**, a well-established hybrid between gradient descent (robust, slow, guaranteed to make progress far from a solution) and Gauss–Newton (fast, locally quadratic convergence, but only reliable close to a solution) — exactly the right tool for a least-squares objective of this shape, which is typically smooth and well-behaved locally but can have a genuinely difficult, high-dimensional landscape globally.

PITFALL

The joint calibration problem is frequently **ill-posed** in the precise technical sense: many different parameter vectors θ fit the observed market prices almost equally well,

particularly for correlation parameters at the long end of the curve where relatively few liquid swaptions constrain the fit. An optimizer run without regularization can therefore return a mathematically valid but economically implausible solution — wildly oscillating correlations, or volatility parameters that fit today’s snapshot but imply an absurd forward volatility surface for tomorrow — purely because that solution happened to sit in the same shallow, flat region of the objective function as the “sensible” answer, with no meaningful difference in fit quality between them. Chapter 8’s SABR β/ρ degeneracy is a small, single-parameter preview of exactly this same phenomenon at much larger scale.

Worked example 18.1 — diagnosing an ill-posed fit.

A calibration run reports a very low total residual (a good headline fit) but the recovered correlation matrix has $\rho_{12,25} = 0.91$ between a 12-year and a 25-year forward rate — higher than the corresponding 2-year/5-year correlation in the same fit, which is economically backwards (distant points on the curve are expected to be *less* correlated than nearby points, per Chapter 14’s decaying parametric form). How should this be diagnosed?

Step 1 — check whether the unconstrained fit even needed that number. Re-run the calibration with the correlation form constrained to Chapter 14’s monotonically-decaying parametric shape (rather than a fully free matrix) and compare the resulting total residual to the original, unconstrained fit.

Step 2 — interpret the comparison. If the constrained fit’s residual is only marginally worse than the unconstrained one, the original $\rho_{12,25} = 0.91$ was not actually needed to explain the market data — it was an artefact of an under-determined optimization landscape, exactly the ill-posedness this chapter describes, and the economically sensible constrained fit should be preferred even though its raw residual is very slightly higher.

Step 3 — if the constrained fit is materially worse. That is a different and more interesting finding: it suggests the swaption market itself is currently pricing in a genuine, real deviation from the usual decaying-correlation pattern (perhaps reflecting a specific, dated policy event expected to affect two particular points on the curve together) — worth flagging to a trader as a market observation, not silently overriding as a numerical artefact.

Interpretation. The headline residual number alone never answers whether a calibrated parameter is economically meaningful; checking it against a deliberately more constrained alternative fit is the standard diagnostic, and Chapter 20 builds this kind of check into the recalibration engine’s validation stage as standard, automated practice rather than an ad-hoc manual investigation.

ON THE DESK

The practical answer most production systems converge on is a hybrid of everything in this chapter: run a fast cascade calibration first, both as a sanity check on the input data and to generate a good starting guess; then run a joint, regularized optimization initialised from that cascade result, using the previous day’s calibrated parameters (or a historically-estimated correlation prior) as the regularization target, so the fit is pulled toward continuity with recent history rather than free to jump to an arbitrary member of the ill-posed solution set purely because of one day’s quoting noise. This combination gets cascade’s speed and diagnosability on the first pass and joint optimization’s better global fit and cross-instrument consistency on the second.

Chapter benchmark

Before moving on, you should be able to, without notes:

1. Explain the cascade calibration idea: process co-terminal swaptions in order so each step has (approximately) one new unknown.
2. State the joint calibration objective function and explain the role of the weights and the regularization term.
3. Explain what it means for a calibration to be ill-posed, and describe a concrete diagnostic (comparing constrained vs. unconstrained fits) for telling an artefact of ill-posedness apart from a genuine market signal.